HashCloak

Code Review and Security Assessment
For
MACI Tally Contract

Initial Delivery: Oct 8th, 2025
Final Delivery: Oct 13th, 2025

Prepared For:
John Guilding | *Ethereum Foundation*

Prepared by:
Manish Kumar | *HashCloak Inc.*

Table Of Contents

Executive Summary

The *Ethereum Foundation's* PSE team engaged HashCloak Inc to perform code review and security assessment for MACI's Tally contract, a core component responsible for aggregating and verifying the results of a MACI-based voting process and funding related infrastructure.

As part of the engagement, HashCloak conducted a comprehensive security review of the Tally contract focused on identifying common security vulnerabilities, best practices, logical errors, and architectural designs that could impact the integrity of vote tallying, fund allocation, and access control mechanisms. We also ran Slither, a smart contract static analyzer tool for detecting vulnerabilities in the solidity smart contract.

Overall, we found the code to be well documented and the implementation adheres to Solidity best practices.

After the initial delivery, the team provided fixes for all the issues in PR#678. The PR was reviewed, and all fixes were confirmed to not introduce any new issues.
We found the issues range from Medium to Informational:

Severity	**Number of Findings**
Critical	0
High	0
Medium	1
Low	0
Informational	2

Scope

For the audit, the given repository was considered:

- https://github.com/privacy-ethereum/maci-platform at commit 853158e4d0e4bef4a4dcad41a2dac7170d99aa19

With following files in scope:

- packages/contracts/contracts/maci/Tally.sol

Overview

Minimal Anti-Collusion Infrastructure (MACI) is an open-source public good that serves as infrastructure for private on-chain voting. It provides privacy and collusion resistance for on-chain voting, both in a quadratic and non-quadratic fashion by using encryption and zero-knowledge proofs (zk-SNARKs) to hide how each person voted while still publicly revealing the final result.

The (in scope) Tally contract serves as the core component responsible for aggregating and verifying the results of a MACI-based voting process. It extends functionality from the TallyBase (from maci-contracts/contracts/Tally.sol) contract and integrates with verifier for finalizing and exposing verifiably correct aggregated voting outcomes while maintaining the privacy.

Methodology

The audit was conducted through a combination of manual verification and using static analyzer tools and cross verifying the solana's common security vulnerabilities, best practices and architectural designs of the tally contract. While the primary focus was the in-scope codebase, we also inspected dependency behavior and interactions with external components such as the Verifier, Registry, and Tally (from maci-contracts/contracts/Tally.sol).
We also used Slither, a smart contract static analyzer tool for detecting any potential vulnerabilities in the smart contract.

Overview of Evaluated Components

Tally.sol

- Access control: use of onlyOwner
- Correct Initialization of contract & preventing re-initialization
- Arithmetic safety
- Error handling and revert conditions
- Data consistency
- Reentrancy and external calls
- Withdrawal authorization
- Any undefined behavior
- Usage of dependencies/import
- Missing access control modifiers for a function
- Initialization, updation and uses of state variables
- Incorrect role-based access control
- General logic errors
- Block Timestamp Manipulation
- Lack of Input Validation

Findings

MACI-1: `Pausable` imported but not used

Type: Medium
Files affected:

- packages/contracts/contracts/maci/Tally.sol

Description: In Tally.sol ,`Pausable` is imported and inherited in the Tally contract but none of the functions in the Tally contract use the associated modifiers. The pause and unpause can be added with onlyOwner modifier for pausing/unpausing contract calls. The Pausable modifier (`whenNotPaused`) can be applied to functions that modify critical state or handle funds, such as `deposit`, `addTallyResults`, and `claim` to allow emergency halting of protocol activity, if needed.

Recommendation: If the intention is to allow pause/unpause then add onlyOwner exposed functions to pause/unpause otherwise the import should be removed to avoid any confusion.

Status: The team wanted to remove the Pausable functionality, so instead of using the associated modifiers, they decided to remove the import. This has been fixed in the following PR: https://github.com/privacy-ethereum/maci-platform/pull/678

MACI-2: `deposit` function allows 0 amount deposits

Type: Informational
Files affected:

- packages/contracts/contracts/maci/Tally.sol

Description: The `deposit()` function allows users to deposit tokens into the contract without verifying that the deposit amount is non-zero. While this does not move any tokens, it still emits a Deposited event and executes a safeTransferFrom() which is unnecessary and redundant.

Recommendation: Add an explicit amount check at the beginning of the function to prevent zero-value deposits:

```
require(amount > 0, "Deposit amount must be greater than zero");
```

Status: The team has provided the fix in this PR:
https://github.com/privacy-ethereum/maci-platform/pull/678

MACI-3: Inconsistency in code comment and MAX_VOICE_CREDITS definition

Type: Informational
Files affected:

- packages/contracts/contracts/maci/Tally.sol

Description: The comment states that MACI allows 2 ** 32 voice credits max but the `MAX_VOICE_CREDITS` constant is hardcoded as 10 ** 9. This creates a discrepancy between documentation and implementation and can create confusion and/or potentially leading to incorrect scaling.

Recommendation: Ensure that MAX_VOICE_CREDITS matches the actual maximum used in MACI core logic. If the true limit is indeed 2 ** 32, the definition should be corrected otherwise, update the comment to reflect the intended value to avoid confusion.

Status: The team has updated the comment in the PR:
https://github.com/privacy-ethereum/maci-platform/pull/678

References

- https://github.com/privacy-ethereum/maci
- https://github.com/crytic/slither
- https://github.com/OpenZeppelin/openzeppelin-contracts
- https://maci.pse.dev/docs/introduction

Appendix

- Result from Slither tool (Clipped) found to be false positive

```
'npx hardhat clean' running (wd: maci-platform/packages/contracts)
'npx hardhat clean --global' running (wd: maci-platform/packages/contracts)
'npx hardhat compile --force' running (wd: maci-platform/packages/contracts)
ERROR:ConvertToIR:Function not found init
ERROR:ContractSolcParsing:Impossible to generate IR for MACI.initPoll
(contracts/maci/MACI.sol#54-60):
 'NoneType' object has no attribute 'type'
ERROR:ConvertToIR:Function not found setRegistry
ERROR:ContractSolcParsing:Impossible to generate IR for MACI.setPollRegistry
(contracts/maci/MACI.sol#65-70):
 'NoneType' object has no attribute 'type'
ERROR:ConvertToIR:Function not found getRegistry
ERROR:ContractSolcParsing:Impossible to generate IR for Tally.init
(contracts/maci/Tally.sol#132-145):
 'NoneType' object has no attribute 'type'
INFO:Detectors:
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-return-in-assembly
INFO:Detectors:
Poll.isInit (node_modules/maci-contracts/contracts/Poll.sol#17) is never
initialized. It is used in:
Poll.numMessages (node_modules/maci-contracts/contracts/Poll.sol#50) is never
initialized. It is used in:
```

```
	- Poll.numSignUpsAndMessages()
(node_modules/maci-contracts/contracts/Poll.sol#258-261)
Tally.tallyResults (node_modules/maci-contracts/contracts/Tally.sol#64) is never
initialized. It is used in:
	- Tally.claim(IPayoutStrategy.Claim) (contracts/maci/Tally.sol#209-242)
	- Tally.getAllocatedAmount(uint256,uint256)
(contracts/maci/Tally.sol#266-279)
Tally.totalTallyResults (node_modules/maci-contracts/contracts/Tally.sol#67) is
never initialized. It is used in:
	- Tally.addTallyResults(ITally.AddTallyResultsArgs)
(contracts/maci/Tally.sol#167-181)
	- Tally.calculateAlpha(uint256) (contracts/maci/Tally.sol#284-300)
Tally.totalSpent (node_modules/maci-contracts/contracts/Tally.sol#70) is never
initialized. It is used in:
	- Tally.calculateAlpha(uint256) (contracts/maci/Tally.sol#284-300)
Tally.token (contracts/maci/Tally.sol#27) is never initialized. It is used in:
	- Tally.deposit(uint256) (contracts/maci/Tally.sol#148-152)
	- Tally.withdraw() (contracts/maci/Tally.sol#155-159)
	- Tally.totalAmount() (contracts/maci/Tally.sol#162-164)
	- Tally.claim(IPayoutStrategy.Claim) (contracts/maci/Tally.sol#209-242)
Tally.registry (contracts/maci/Tally.sol#30) is never initialized. It is used in:
	- Tally.addTallyResults(ITally.AddTallyResultsArgs)
(contracts/maci/Tally.sol#167-181)
	- Tally.claim(IPayoutStrategy.Claim) (contracts/maci/Tally.sol#209-242)
Tally.maxCap (contracts/maci/Tally.sol#33) is never initialized. It is used in:
	- Tally.getAllocatedAmount(uint256,uint256)
(contracts/maci/Tally.sol#266-279)
Tally.voiceCreditFactor (contracts/maci/Tally.sol#36) is never initialized. It is
used in:
	- Tally.getAllocatedAmount(uint256,uint256)
(contracts/maci/Tally.sol#266-279)
	- Tally.calculateAlpha(uint256) (contracts/maci/Tally.sol#284-300)
Tally.cooldown (contracts/maci/Tally.sol#39) is never initialized. It is used in:
Tally.custodian (contracts/maci/Tally.sol#42) is never initialized. It is used in:
	- Tally.withdraw() (contracts/maci/Tally.sol#155-159)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#uninitialized-state-v
ariables



INFO:Detectors:
2 different versions of Solidity are used:
- Version constraint ^0.8.20 is used by:
- Version constraint ^0.8.10 is used by:
```

```
INFO:Detectors:
Tally.addTallyResult(uint256,uint256,uint256[][],uint256,uint256,uint256,uint8)
(contracts/maci/Tally.sol#184-206) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code


Tally.cooldown (contracts/maci/Tally.sol#39) should be constant
Tally.custodian (contracts/maci/Tally.sol#42) should be constant
Tally.maxCap (contracts/maci/Tally.sol#33) should be constant
Tally.registry (contracts/maci/Tally.sol#30) should be constant
Tally.token (contracts/maci/Tally.sol#27) should be constant
Tally.totalSpent (node_modules/maci-contracts/contracts/Tally.sol#70) should be
constant
Tally.totalTallyResults (node_modules/maci-contracts/contracts/Tally.sol#67) should
be constant
Tally.voiceCreditFactor (contracts/maci/Tally.sol#36) should be constant
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-
could-be-declared-constant
```